

AIGC 人物异常检测 - 完整复现报告

版本: v1.0
运行环境: Ubuntu / Python 3.10 / PyTorch 2.4.0+cu121 / CUDA 可用
Conda 环境: aigc
数据集来源: HumanRefiner (MIT License)

目录

- 1. 项目概述
- 2. 环境依赖
- 3. 数据集准备
- 4. mmdet+mmpose+SAM 合成数据流程
- 5. 模型设计
- 6. 训练流程 (sk_resnet.py)
- 7. 域自适应微调 (domain_fine_tuning.py)
- 8. 测试评估 (eval.py)
- 9. 推理调试 (debug.py)
- 10. 实际测试结果
- 11. 完整复现步骤

1. 项目概述

1.1 项目目标

本项目旨在自动识别 AI 生成内容 (AIGC) 中人物图片的质量缺陷。AIGC 生成的图片中，人物常常出现各种异常 (如手部畸形、肢体缺失等)，算法可以对这些图片进行快速分类和异常检测，从而在批量生成场景中自动过滤掉质量较差的图片。

核心检测能力:

- 判断一张含人物的图片是"正常"还是"异常"
- 如果异常，进一步识别属于哪种类型的异常 (8 类)

支持的检测场景 (8 类异常):

编号	异常类型	说明
1	手部畸形	手指数量不对、手指融合等
2	肢体缺失	手臂或腿缺少
3	肢体增加	多出多余的手臂或腿
4	肢体模糊	肢体部分模糊不清
5	脸部畸形	面部比例失调、五官错位
6	脸部模糊	面部不够清晰
7	身体畸形	身体比例异常、身体结构不合理
8	其他	未归入上述类别的异常

1.2 项目文件结构

aigcAnomalyDetect/	
├─ config.yaml	# 全局配置文件（路径 + 超参）
├─ sk_resnet.py	# 主训练脚本（DDP/单卡, 40 epoch, EMA）
├─ domain_fine_tuning.py	# 域自适应微调脚本（防止灾难性遗忘）
├─ eval.py	# 测试评估脚本（阈值优化 + 验证/测试集评估）
├─ debug.py	# 推理调试脚本（可视化中间过程）
├─ sk_dataset.py	# 数据集类 + JointTransform + FocalLoss
├─ loss_calculator.py	# 损失函数计算器（多任务损失 + 阈值优化）
├─ models/	
│ └─ ConvNeXt.py	# 主模型 ConvNeXtTinyFusion
│ └─ fs_resnet.py	# 备选模型 MultiScaleFusionResNet (ResNet50 版)
├─ dataSet/	
│ └─ dataset.py	# ImageWithLabelDataset (单标签二分类)
│ └─ ml_dataset.py	# ImageWithMultiLabelDataset (多标签)
├─ evaluate/	
│ └─ evaluate.py	# 评估指标计算（calc_metrics_balance 等）
│ └─ ml_evaluate.py	# 多标签评估（evalModel_multiple）
├─ mmpose_sk.py	# 人体/手部关键点检测 + 骨架热力图生成
├─ sam_fusion.py	# SAM 手部 patch 融合到目标图（合成数据）
├─ dataWash.py	# 数据集清洗脚本（标签统计 + 过滤）
├─ rearrange_data.py	# 数据集合并重组（7:2:1 划分）
├─ rearrange_finetune_data.py	# 微调数据集重组
├─ data/	
│ └─ rearranged_train/	# 训练集（50,459 张）
│ └─ rearranged_val/	# 验证集（14,417 张）
│ └─ rearranged_test/	# 测试集（7,209 张）
│ └─ fusion_result_*/	# 合成数据目录
│ └─ huawei_data/	# 华为域数据（微调用）
│ │ └─ train/	# 训练域（248 张）
│ │ └─ test/	# 测试域（248 张）
│ └─ crowdpose_trainval.json	# CrowdPose 数据索引
├─ models_parameter/	
│ └─ resnet/	
│ │ └─ best_model_opt.pth	# 最佳模型权重
│ │ └─ best_model_ema_opt.pth	# 最佳 EMA 模型权重
│ │ └─ last_model_opt.pth	# 最后一轮模型权重
│ └─ finetuned/	
│ └─ cross_domain_fine_tuned.pth	# 域自适应微调后权重
├─ sam_vit_h_4b8939.pth	# SAM ViT-H 模型权重（2.5 GB）
├─ mmpose/	# MMPose 代码库
├─ segment-anything/	# SAM 代码库
├─ fusion_test/	# 合成数据参考实现
│ └─ fusion.py	# 仿射变换对齐逻辑参考
├─ debug/	# 推理调试输出目录
├─ docs/	# 报告文档
│ └─ model.md	# 汇报模板
│ └─ 项目报告.md	# 面向领导的报告
│ └─ 项目复现报告.md	# 本文件
└─ README.md	# 项目说明

2. 环境依赖

2.1 核心依赖

依赖	版本	说明
Python	3.10	编程语言
PyTorch	2.4.0+cu121	深度学习框架
CUDA	可用	GPU 加速
torchvision	>= 0.19	图像处理和模型
OpenMMLab MMPose	0.10.7	人体关键点检测
MMDetection	3.3.0	目标检测
MMCV	2.1.0 (mmpose_sk.py 中伪装为 2.1.0)	基础框架
NumPy	-	数值计算
Pillow	-	图像处理
Matplotlib	-	可视化
OpenCV	-	图像操作
tqdm	-	进度条
PyYAML	-	配置文件解析

2.2 安装步骤

```
# 使用 conda 环境
conda create -n aigc python=3.10 -y
conda activate aigc

# 安装 PyTorch (示例, 根据 CUDA 版本调整)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# 安装 OpenMMLab 系列
pip install mmpose==0.10.7
pip install mmdet==3.3.0
pip install mmcv==2.1.0

# 安装 Segment Anything
pip install git+https://github.com/facebookresearch/segment-anything.git

# 其他依赖
pip install numpy Pillow matplotlib tqdm pyyaml opencv-python
```

2.3 下载预训练模型

项目代码中会自动从以下地址下载预训练权重：

模型	配置路径	
RTMDet-Person	mmpose/projects/rtmpose/rtmdet/person/rtmdet_m_640-8xb32_coco-person.py	rtmdet_m_8xb32-100e_coc
RTMPose-L Body (COCO 17 keypoints)	mmpose/projects/rtmpose/rtmpose/body_2d_keypoint/rtmpose-1_8xb256-420e_coco-256x192.py	rtmpose-1_simcc-aic-coc
RTMDet-Hand	mmpose/projects/rtmpose/rtmdet/hand/rtmdet_nano_320-8xb32_hand.py	rtmdet_nano_8xb32-300e_

模型	配置路径	
RTMPose-M Hand (COCO WHOLEBOD Y 21 keypoints)	mmpose/projects/rtmpose/rtmpose/hand_2d_keypoint/rtmpose-m_8xb32-210e_coco-wholebody-hand-256x256.py	rtmpose-m_simcc-hand5_p

SAM 模型： 需手动下载 sam_vit_h_4b8939.pth （2.5 GB）到项目根目录。

3. 数据集准备

3.1 原始数据集

- **HumanRefiner 数据集**（MIT License）：包含大量人工标注的 AIGC 人物图片
- 原始人体异常类型分为 7 类：头部异常、脖子异常、身体异常、手臂异常、手异常、腿异常、脚异常
- 本项目去除 HumanRefiner 原始数据集的 bbox 属性，仅使用数据标签

3.2 异常分类体系

编号	异常类型	说明
1	手部畸形	手指数量不对、手指融合等
2	肢体缺失	手臂或腿缺少
3	肢体增加	多出多余的手臂或腿
4	肢体模糊	肢体部分模糊不清
5	脸部畸形	面部比例失调、五官错位
6	脸部模糊	面部不够清晰
7	身体畸形	身体比例异常、身体结构不合理
8	其他	未归入上述类别的异常

3.3 数据清洗流程（dataWash.py）

原始数据 → 标签合并 → 过滤非人类 → 统计分布

核心函数：

函数	功能	输出
find_label_files()	将多异常标签合并为单一异常标签 1，正常标签为 0	washed_labels/
find_multiple_label_files()	保留多标签格式（如 1,3,7）	washed_multiple_labels/
label_count()	统计各异常类别数量	控制台输出

标签规则：

- 异常编号 1-8 表示对应异常类型
- 编号 9 表示非人类样本（过滤）
- 一个文件可以有多行，每行一个异常编号
- 同时存在多个异常编号表示多标签样本

3.4 数据集重组流程（rearrange_data.py）

原始数据 → 添加前缀 → 合并所有数据 → 7:2:1 随机划分 → 软链接重组

重组步骤：

- 1. **添加前缀**：为每个文件添加源文件夹前缀（ train_xxx.png ， val_xxx.png ， crowdpose_xxx.png ），防止文件名冲突
- 2. **合并打乱**：将 train + val + crowdpose 所有数据合并并随机打乱
- 3. **7:2:1 划分**：
 - 训练集：前 70%
 - 验证集：70%-90%
 - 测试集：后 10%
- 4. **软链接**：使用 os.symlink() 创建软链接，不复制文件

重组后的目录结构：

```
rearranged_train/
├─ washed_images/           # 图片文件（JPG/PNG），带前缀
├─ washed_multiple_labels/  # 多标签文件（*.txt）
│   └─ train_xxx.txt        # 内容为 "1,3,7" 或 "0"
│   └─ ...
└─ cache_skeleton/          # 骨架热力图缓存（自动计算）
    └─ cache_body/          # 人体骨架缓存（*.pt）
        └─ cache_hand/      # 手部骨架缓存（*.pt）
```

标签格式：

- 正常图片：标签文件内容为 0
- 异常图片：标签文件内容为 1,3,7 （表示同时存在第 1、3、7 类异常）

3.5 数据集统计

数据集	图片数	正常	异常	主要异常分布
训练集	50,459	21,036	29,423	5:27,266, 1:5,356, 7:4,214, 4:3,338, 6:2,095, 8:709, 3:654, 2:129
验证集	14,417	6,098	8,319	5:7,687, 1:1,449, 7:1,209, 4:948, 6:634, 8:199, 3:178, 2:30
测试集	7,209	2,982	4,227	5:3,912, 1:756, 7:633, 4:496, 6:348, 8:128, 3:97, 2:25
华为域 train	248	124	124	1:124（简化为 2 类）
华为域 test	248	125	123	1:123（简化为 2 类）

说明：

- 主数据集（rearranged_*）包含 8 类异常，其中第 5 类（脸部畸形/身体畸形）占绝大多数
- 华为域数据简化为 2 类：normal 和 1（异常），用于域自适应微调评估

3.6 数据增强策略

增强操作	训练集	验证/测试集
Resize	224x224	224x224
水平翻转	概率 0.5	无
随机旋转	5 度, 概率 0.5	无
亮度调整	5%, 概率 0.5	无
对比度调整	5%, 概率 0.5	无
归一化	ImageNet stats [0.485,0.456,0.406] / [0.229,0.224,0.225]	ImageNet stats

增强代码实现（sk_dataset.py · JointTransform）：

```
class JointTransform:
    def __init__(self, input_size=224, hflip_prob=0.5, rotate_deg=5,
                 rotate_prob=0.5, brightness=0.05, contrast=0.05,
                 brightness_prob=0.5, contrast_prob=0.5):
        # ...

    def __call__(self, image, skeleton, hand_skeleton):
        # 1. Resize
        image = TF.resize(image, (self.input_size, self.input_size))

        # 2. Random Flip
        if random.random() < self.hflip_prob:
            image = TF.hflip(image)
            skeleton = torch.flip(skeleton, dims=[2])      # 水平翻转人体骨架
            hand_skeleton = torch.flip(hand_skeleton, dims=[2]) # 水平翻转手部骨架

        # 3. Random Rotate（对 RGB、人体骨架、手部骨架同时旋转）
        if self.rotate_deg > 0 and random.random() < self.rotate_prob:
            angle = random.uniform(-self.rotate_deg, self.rotate_deg)
            image = TF.rotate(image, angle, interpolation=InterpolationMode.BILINEAR)
            skeleton = TF.rotate(skeleton.unsqueeze(0), angle,
                               interpolation=InterpolationMode.BILINEAR).squeeze(0)
            hand_skeleton = TF.rotate(hand_skeleton.unsqueeze(0), angle,
                                     interpolation=InterpolationMode.BILINEAR).squeeze(0)

        # 4. Brightness & Contrast
        if random.random() < self.brightness_prob:
            brightness_factor = random.uniform(max(0, 1 - self.brightness), 1 + self.brightness)
            image = TF.adjust_brightness(image, brightness_factor)

        if random.random() < self.contrast_prob:
            contrast_factor = random.uniform(max(0, 1 - self.contrast), 1 + self.contrast)
            image = TF.adjust_contrast(image, contrast_factor)

        # 5. RGB to Tensor + Normalize
        image = TF.to_tensor(image)
        image = TF.normalize(image, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

        return image, skeleton, hand_skeleton
```

3.7 骨架热力图生成

骨架热力图是输入的第 3、4 通道，由 OpenMMLab RTMPose 系列生成。

生成流程：

原始图像 → RTMPose 人体检测 → 人体关键点 → 高斯模糊 → 人体骨架热力图(224x224)
原始图像 → RTMPose 手部检测 → 手部关键点 → 高斯模糊 → 手部骨架热力图(224x224)

骨架热力图生成参数：

类型	模型	关键点数量	高斯核大小	阈值
人体骨架	RTMPose-L (COCO 17 keypoints)	17	kernel=45	0.4
手部骨架	RTMPose-M (COCO WHOLEBODY 21 keypoints)	21	kernel=25	0.2

骨架热力图缓存机制：

骨架热力图计算较慢，首次加载数据集时会缓存为 .pt 文件到 cache_skeleton/ 目录，后续直接从缓存加载：

```
cache_skeleton/
├── cache_body/           # 人体骨架缓存
│   ├── train_xxx.pt
│   └── ...
└── cache_hand/          # 手部骨架缓存
    ├── train_xxx.pt
    └── ...
```

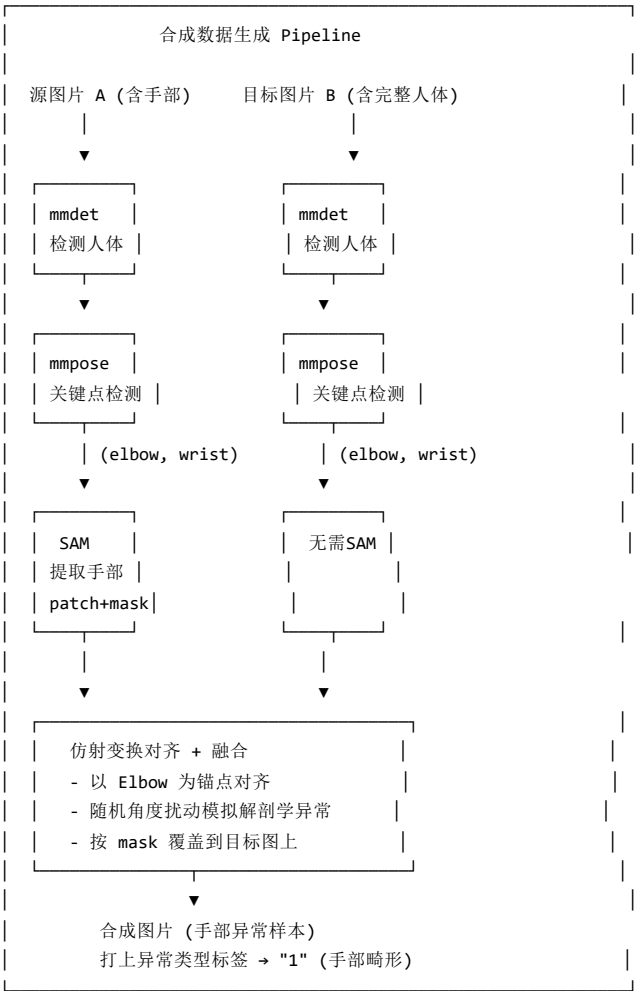
缓存文件注释掉了自动保存逻辑（代码中 `torch.save` 行被注释），实际使用时需要手动开启。

4. mmdet+mmpose+SAM 合成数据流程

4.1 合成数据原理

除了人工标注数据外，本项目还通过 **mmdet + mmpose + SAM** 三阶段 pipeline 合成带有手部异常的数据样本，用于扩充手部畸形类别的训练数据。

合成原理：



4.2 详细流程

步骤	工具	输入	输出	说明
1. 人体检测	mmdet (RTMDet-Person)	原始图像	人体 bounding box	筛选置信度 > 0.4 的检测结果

步骤	工具	输入	输出	说明
2. 姿态估计	mmpose (RTMPose-L Body)	人体框	elbow/wrist 关键点坐标	COCO 17 keypoints, 左侧 elbow=7/wrist=9, 右侧 elbow=8/wrist=10
3. 手部分割	SAM (Segment Anything)	关键点提示 (wrist, elbow)	手部 mask + patch	使用 point prompt 模式 (wrist 为正样本, elbow 为负样本), multimask_output=True
4. 仿射变换	自定义算法	源 patch+mask, 目标关键点	变换后 patch+mask	以 elbow 为锚点, 向量对齐 + 随机角度扰动 ($\pm 15^\circ$)
5. 图像融合	自定义算法	目标图+变换后 patch	合成图片	按 mask 布尔索引覆盖, 保留目标图其他部分

4.3 仿射变换对齐逻辑

核心代码位于 sam_fusion.py 的 merge_hand_to_target() 函数：


```

def merge_hand_to_target(target_img_path, patch_img, patch_mask,
                        target_points, src_kpts, patch_offset):
    """
    以 Elbow 为锚点进行对齐和旋转

    Args:
        target_img_path: 目标图片路径
        patch_img: 源 patch 图片 (含手部)
        patch_mask: 源 patch 的 mask
        target_points: 目标位置的关键点 {'wrist': [x,y], 'elbow': [x,y]}
        src_kpts: 源图片的关键点 {'wrist': [x,y], 'elbow': [x,y]}
        patch_offset: 从 SAM 提取时记录的 patch 偏移 [x_min, y_min]
    """

    # 1. 动态增加 Padding (防止旋转裁剪)
    pad = max(patch_img.shape[:2])
    patch_img = cv2.copyMakeBorder(patch_img, pad, pad, pad, pad,
                                   cv2.BORDER_CONSTANT, value=0)
    patch_mask = cv2.copyMakeBorder(patch_mask, pad, pad, pad, pad,
                                    cv2.BORDER_CONSTANT, value=0)

    h, w = patch_img.shape[:2]

    # 2. 计算 Elbow 在 Patch 内部的局部坐标 (Anchor)
    local_elbow = (float(src_kpts['elbow'][0] - patch_offset[0] + pad),
                  float(src_kpts['elbow'][1] - patch_offset[1] + pad))

    # 3. 向量对齐与变换参数计算
    # 源向量 (Elbow -> Wrist)
    vec_src = np.array(src_kpts['wrist']) - np.array(src_kpts['elbow'])
    src_angle = math.atan2(vec_src[1], vec_src[0])

    # 目标向量 (Elbow -> Wrist)
    vec_dst = np.array(target_points['wrist']) - np.array(target_points['elbow'])
    dst_len = np.linalg.norm(vec_dst)
    dst_angle = math.atan2(vec_dst[1], vec_dst[0])

    # 【关键】随机扰动角度 (模拟解剖学异常)
    angle_offset = math.radians(np.random.randint(-15, 15))
    final_angle = (dst_angle - src_angle) + angle_offset

    scale_ratio = dst_len / np.linalg.norm(vec_src)

    # 4. 执行仿射变换
    M = cv2.getRotationMatrix2D(local_elbow, math.degrees(final_angle), scale_ratio)
    patch_rotated = cv2.warpAffine(patch_img, M, (w, h), flags=cv2.INTER_LINEAR)
    mask_rotated = cv2.warpAffine(patch_mask, M, (w, h), flags=cv2.INTER_NEAREST)

    # 5. 精确坐标重映射
    p = np.array([local_elbow[0], local_elbow[1], 1])
    rotated_elbow_pos = M @ p

    # 计算将 Patch 放置在 Target 上所需的左上角偏移
    dx = int(target_points['elbow'][0] - rotated_elbow_pos[0])
    dy = int(target_points['elbow'][1] - rotated_elbow_pos[1])

    # 6. 高效矩阵切片覆盖 (NumPy 布尔索引)
    tx1, ty1 = max(0, dx), max(0, dy)
    tx2, ty2 = min(img_w, dx+w), min(img_h, dy+h)
    px1, py1 = tx1 - dx, ty1 - dy
    px2, py2 = px1 + (tx2 - tx1), py1 + (ty2 - ty1)

    if tx1 < tx2 and ty1 < ty2:
        roi = target_img[ty1:ty2, tx1:tx2]
        sub_patch = patch_rotated[py1:py2, px1:px2]
        sub_mask = mask_rotated[py1:py2, px1:px2]

```

```
if len(sub_mask.shape) == 3:
    sub_mask = cv2.cvtColor(sub_mask, cv2.COLOR_BGR2GRAY)

# 只有当 Mask > 127 时才替换目标图内容
roi[sub_mask > 127] = sub_patch[sub_mask > 127]
target_img[ty1:ty2, tx1:tx2] = roi

return target_img, final_center
```

关键设计说明：

1. **Elbow 锚点**：选择肘部作为对齐锚点，因为肘部是人体关节，位置相对稳定
2. **向量对齐**：通过 `elbow→wrist` 向量计算旋转角度和缩放比例，使融合的手部尺寸与目标手部匹配
3. **随机角度扰动 ($\pm 15^\circ$)**：这是关键设计—— $\pm 15^\circ$ 的扰动可以模拟手部位置不正常的解剖学异常，使合成的样本更接近真实的手部畸形场景
4. **Padding**：旋转时会裁剪图像，所以先动态增加 Padding 防止信息丢失
5. **坐标重映射**：旋转后需要重新计算 patch 在目标图上的位置
6. **高效覆盖**：使用 NumPy 布尔索引 `roi[sub_mask > 127] = sub_patch[sub_mask > 127]`，比循环高效得多

4.4 SAM 提取手部 Patch

```
def create_asset_library_and_fuse(image_path, source_points,
                                target_img_path, target_points, save_dir=None):
    """
    直接从 pose 结果运行 SAM 提取手部 patch 和 mask，然后融合到目标图上

    Args:
        image_path: 源图片路径 (用于提取手部 patch)
        source_points: {'wrist': [x,y], 'elbow': [x,y]} 源图片关键点 (用于 SAM 提示)
        target_img_path: 目标图片路径 (用于融合)
        target_points: {'wrist': [x,y], 'elbow': [x,y]} 目标位置关键点 (用于融合对齐)
        save_dir: 可选，保存最终结果
    """
    image = cv2.imread(image_path)
    predictor.set_image(image)

    # SAM 预测: wrist 为正样本(1), elbow 为负样本(0)
    input_point = np.array([source_points['wrist'], source_points['elbow']])
    input_label = np.array([1, 0])

    masks, scores, _ = predictor.predict(
        point_coords=input_point,
        point_labels=input_label,
        multimask_output=True,
    )

    best_mask = masks[np.argmax(scores)].astype(np.uint8) * 255

    # 寻找最小外接矩形进行紧密裁剪
    coords = np.argwhere(best_mask > 0)
    y_min, x_min = coords.min(axis=0)
    y_max, x_max = coords.max(axis=0)

    cropped_patch = image[y_min:y_max+1, x_min:x_max+1]
    cropped_mask = best_mask[y_min:y_max+1, x_min:x_max+1]

    # 计算元数据
    bone_length = float(np.linalg.norm(
        np.array(source_points['wrist']) - np.array(source_points['elbow'])))
    metadata = {
        "id": os.path.basename(image_path).split('.')[0],
        "bone_length": bone_length,
        "patch_offset": [x_min, y_min],
        "source_points": source_points,
        "target_points": target_points
    }

    result, final_center = merge_hand_to_target(
        target_img_path, cropped_patch, cropped_mask,
        target_points, source_points, metadata['patch_offset']
    )

    if result is not None and save_dir:
        save_path = os.path.join(save_dir, f"result_{metadata['id']}.jpg")
        cv2.imwrite(save_path, result)

    return result, metadata
```

4.5 合成数据标注

文件名	标签文件	标签内容	说明
fusion_result/washed_images/result_xxx.jpg	result_xxx.txt	1	手部畸形样本（异常）

合成数据输出目录：

目录	内容
data/fusion_result_1/	第 1 批次合成数据
data/fusion_result_2/	第 2 批次合成数据
data/fusion_result_3/	第 3 批次合成数据

每个目录包含：

- washed_images/ — 合成后的图片 (JPG)
- washed_multiple_labels/ — 对应的标签文件 (TXT)

4.6 合成代码文件

文件	功能
sam_fusion.py	核心合成逻辑：mmdet+mmpose+SAM 完整 pipeline
fusion_test/fusion.py	合成参考实现（含仿射变换和对齐逻辑，带 JSON 资产库）
mmpose_sk.py	人体/手部关键点检测和骨架热力图生成
sam.py	SAM 资产库构建脚本

4.7 SAM 关键点提示说明

SAM 使用 point prompt 模式提取手部：

点	坐标来源	标签	作用
手腕 (wrist)	RTMPose 关键点	1（前景）	告诉 SAM "这里是要分割的区域"
肘部 (elbow)	RTMPose 关键点	0（背景）	告诉 SAM "这里不是要分割的区域"

这样 SAM 能准确分割出手臂/手部区域，排除身体其他部分。

4.8 仿射变换完整参数

参数	计算公式	说明
旋转角度	dst_angle - src_angle + random(-15°, 15°)	向量对齐 + 随机扰动
缩放比例	dst_length / src_length	源向量与目标向量长度比
锚点	源 elbow 坐标	以肘部为旋转中心
Padding	max(patch_width, patch_height)	防止旋转裁剪

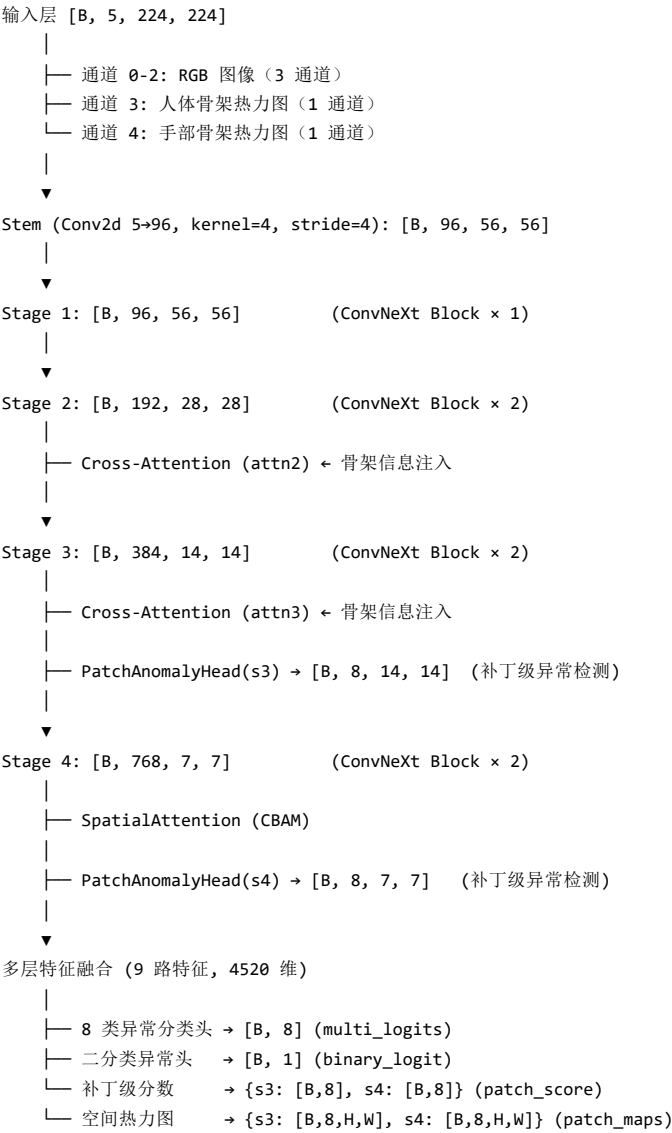
5. 模型设计

5.1 主模型：ConvNeXtTinyFusion

文件位置： models/ConvNeXt.py

5.1.1 整体架构

本模型基于 ConvNeXt Tiny（ImageNet 预训练）进行多层次改造，核心设计思路是将 CNN 的空间感知能力与骨骼关键点的结构先验相结合。



Backbone 网络结构：

Stage	输出尺寸	通道数	模块组成
Stem	56×56	96	Conv2d(5→96, stride=4) + LayerNorm + GELU
Stage 1	56×56	96	ConvNeXt Block
Stage 2	28×28	192	ConvNeXt Block × 2 + Cross-Attention 注入
Stage 3	14×14	384	ConvNeXt Block × 2 + Cross-Attention 注入
Stage 4	7×7	768	ConvNeXt Block × 2 + SpatialAttention

5.1.2 Stem 改造（3 通道 → 5 通道）

原始 ConvNeXt 接受 3 通道 RGB 输入，本项目将其扩展为 5 通道，引入人体结构先验：

```
# 原始 ConvNeXt Stem 输入为 3 通道 RGB，扩展为 5 通道
base_model = convnext_tiny(weights=ConvNeXt_Tiny_Weights.IMAGENET1K_V1)
old_conv = base_model.features[0][0] # 原始 Conv2d(3, 96)

self.stem_conv = nn.Conv2d(5, 96, kernel_size=4, stride=4)
with torch.no_grad():
    # RGB 通道继承预训练权重
    self.stem_conv.weight[:, :3, :, :] = old_conv.weight
    self.stem_conv.bias = old_conv.bias
    # 骨架热力图通道初始化为 0（通过微调学习）
    self.stem_conv.weight[:, 3:, :, :] = 0.0
```

输入 5 通道详细说明：

通道	来源	说明
0-2	RGB 图像	原始彩色图像，通过 PIL 读取后标准化
3	人体骨架热力图	通过 OpenMMLab RTMPose 检测全身关键点，经高斯模糊(kernel=45)生成概率图
4	手部骨架热力图	通过 RTMPose 检测手部关键点，经高斯模糊(kernel=25)生成概率图

骨架热力图生成流程：

原始图像 → RTMPose 人体检测 → 关键点坐标 → 高斯模糊 → 连续概率热力图（224x224）
原始图像 → RTMPose 手部检测 → 关键点坐标 → 高斯模糊 → 连续概率热力图（224x224）

5.1.3 交叉注意力注入层（StandardCrossAttention）

设计动机： CNN 的特征图本身对人体的结构位置不敏感，通过 Cross-Attention 将骨骼关键点信息注入到中间特征层，引导模型在特征空间中关注人物的关键部位（手、臂、腿等）。

```
class StandardCrossAttention(nn.Module):
    def __init__(self, dim, sk_dim=2, dropout=0.2):
        super().__init__()
        self.inter_dim = dim // 8      # 降维到 24/48
        self.scale = self.inter_dim ** -0.5

        self.q = nn.Conv2d(dim, self.inter_dim, 1)      # Q 来自特征图
        self.k = nn.Conv2d(sk_dim, self.inter_dim, 1)   # K 来自骨架图
        self.v = nn.Conv2d(sk_dim, dim, 1)             # V 来自骨架图
        self.gamma = nn.Parameter(torch.zeros(1))       # 可学习缩放系数
        self.dropout = nn.Dropout2d(dropout)            # Dropout 防止过拟合

    def forward(self, x, sk):
        B, C, H, W = x.shape
        # 1. 对齐骨架图尺寸
        sk_res = F.interpolate(sk, size=(H, W), mode='bilinear', align_corners=False)

        # 2. 生成 Q, K, V
        proj_query = self.q(x).view(B, self.inter_dim, -1).permute(0, 2, 1) # [B, N, C']
        proj_key   = self.k(sk_res).view(B, self.inter_dim, -1)            # [B, C', N]
        proj_value = self.v(sk_res).view(B, C, -1)                        # [B, C, N]

        # 3. 计算注意力矩阵
        energy = torch.bmm(proj_query, proj_key) * self.scale # [B, N, N]
        attention = F.softmax(energy, dim=-1)                # 对空间像素归一化

        # 4. 注意力加权叠加
        out = torch.bmm(proj_value, attention.permute(0, 2, 1))
        out = out.view(B, C, H, W)
        out = self.dropout(out)

        # 5. 残差连接: 将骨架注意力引导的信息注入原特征
        return x + self.gamma * out
```

设计参数：

参数	值	说明
inter_dim	C // 8	降维后的注意力维度，减少计算量（192→24, 384→48）
sk_dim	2	骨骼通道数（人体骨架 + 手部骨架）
dropout	0.2-0.3	Dropout2d 防止过拟合
gamma	可学习参数	初始化 0，训练后自动调节骨骼信息的注入强度

注入位置：

注入点	Stage 输出	特征尺寸	通道数	说明
attn2	Stage 2 后	28×28	192	注入第一次骨骼信息
attn3	Stage 3 后	14×14	384	注入第二次骨骼信息

随网络加深，骨骼信息与特征融合越来越充分。

5.1.4 空间注意力模块（CBAM Spatial Attention）

```
class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super().__init__()
        assert kernel_size in (3, 7)
        padding = 3 if kernel_size == 7 else 1
        # 输入 2 通道 (Max 和 Avg 合并)，输出 1 通道权重
        self.conv1 = nn.Conv2d(2, 1, kernel_size, padding=padding, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # 1. 通道维度取 Max → [B, 1, H, W]
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        # 2. 通道维度取 Avg → [B, 1, H, W]
        avg_out = torch.mean(x, dim=1, keepdim=True)
        # 3. 拼接 → [B, 2, H, W]
        x_cat = torch.cat([avg_out, max_out], dim=1)
        # 4. 卷积 + Sigmoid 生成权重图 → [B, 1, H, W]
        out = self.conv1(x_cat)
        return self.sigmoid(out)
```

该模块学习"图像中哪里是重要的"，增强模型对异常区域的空间感知能力。应用在 Stage 4 (768ch, 7×7) 之后。

5.1.5 Patch 级异常检测头（PatchAnomalyHead）

在 Stage 3 和 Stage 4 分别接一个补丁级异常检测头，实现像素级（更精确地说是 patch 级）的异常定位：

```
class PatchAnomalyHead(nn.Module):
    def __init__(self, in_dim, num_classes):
        super().__init__()
        self.head = nn.Sequential(
            nn.Conv2d(in_dim, in_dim // 2, 3, padding=1),
            nn.GELU(),
            nn.Conv2d(in_dim // 2, num_classes, 1)
        )

    def forward(self, x):
        return self.head(x) # [B, K, H, W]
```

LSE 聚合（Log-Sum-Exp）防止 exp 溢出：

```
def lse_aggregate(patch_logits, r=10):
    """
    将空间维度 [H, W] 聚合为 [B, K]
    r=10 为温度系数，比简单 Max/Avg Pool 能更好地捕捉异常区域的峰值响应
    """
    B, K, H, W = patch_logits.shape
    x = patch_logits.view(B, K, -1)

    # Log-Sum-Exp Trick: 减去最大值防止 exp 溢出
    x_max, _ = torch.max(x, dim=-1, keepdim=True)
    lse = x_max + (1.0 / r) * torch.log(
        torch.mean(torch.exp(r * (x - x_max)), dim=-1, keepdim=True)
    )
    return lse.squeeze(-1) # [B, K]
```

Patch 级输出：

输出	维度	说明
patch_maps['s3']	[B, 8, 14, 14]	Stage 3 补丁级异常 logits

输出	维度	说明
patch_maps['s4']	[B, 8, 7, 7]	Stage 4 补丁级异常 logits
patch_score['s3']	[B, 8]	Stage 3 经 LSE 聚合后的分数
patch_score['s4']	[B, 8]	Stage 4 经 LSE 聚合后的分数

5.1.6 多层特征融合策略

最终分类器融合来自网络不同层次、不同聚合方式的 **9 路特征**：

特征来源	维度	说明
neck feat	768	Stage4 经 SpatialAttention 后，可学习权重融合（ $\alpha_{Avg} + \beta_{Max}$ ），再经 MLP(768→768) + LayerNorm + GELU + Dropout(0.4)
S3 Avg Pool	384	Stage3 全局平均池化
S3 Max Pool	384	Stage3 全局最大池化
S4 Avg Pool	768	Stage4 全局平均池化
S4 Max Pool	768	Stage4 全局最大池化（经 Dropout 0.2）
Patch S3 Avg	8	PatchAnomalyHead S3 的 LSE 聚合
Patch S3 Max	8	PatchAnomalyHead S3 的 Max 聚合
Patch S4 Avg	8	PatchAnomalyHead S4 的 LSE 聚合
Patch S4 Max	8	PatchAnomalyHead S4 的 Max 聚合
合计	4520	$768 + 384 \times 2 + 768 \times 2 + 8 \times 4 = 4520$

可学习融合权重：

```

self.fusion_weights = nn.Parameter(torch.ones(2))

# 通过 softmax 自动学习 Avg 和 Max 的权重比例
weights = F.softmax(self.fusion_weights, dim=0)
alpha, beta = weights[0], weights[1]

feat_avg = F.adaptive_avg_pool2d(x, (1, 1)).flatten(1) # [B, 768]
feat_max = F.adaptive_max_pool2d(x_refined, (1, 1)).flatten(1) # [B, 768]
feat_max = self.max_pool_dropout(feat_max)

feat = alpha * feat_avg + beta * feat_max # 加权融合
feat = self.neck_projector(feat) # MLP(768→768) + LayerNorm + GELU + Dropout(0.4)
feat = self.final_norm(feat) # LayerNorm(eps=1e-6)

```

5.1.7 多任务输出头

```

self.combined_dim = 4520 # 768 + 384×2 + 768×2 + 8×4
self.combined_multi_classifier = nn.Linear(self.combined_dim, 8)
self.combined_binary_classifier = nn.Linear(self.combined_dim, 1)

```

模型同时输出 4 种预测结果：

输出	维度	作用
multi_logits	[B, 8]	8 类异常的独立预测分数
binary_logit	[B, 1]	整体异常/正常的二分类分数

输出	维度	作用
patch_score	{s3: [B,8], s4: [B,8]}	补丁级聚合分数，支持可解释性
patch_maps	{s3: [B,8,H,W], s4: [B,8,H,W]}	空间热力图，可视化模型关注区域

5.1.8 损失函数

采用多任务联合优化：

```
# 总损失
total_loss = loss_multi + 5.0 * loss_binary + 0.0 * loss_patch
```

Loss	权重	说明
Focal Loss （多标签）	1.0	解决 8 类异常的类别不平衡问题，gamma=2.5
Focal Loss （二分类）	5.0	整体异常/正常分类，放大二分类损失以加强监督信号
Patch Loss	0.0	可选，当前未启用，未来可通过调整 loss_patch_para 开启

Focal Loss 实现：

```
class FocalLoss(nn.Module):
    def __init__(self, alpha=None, gamma=2.0, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.alpha = alpha # 正样本权重
        self.gamma = gamma # 聚焦参数

    def forward(self, logits, targets):
        bce_loss = F.binary_cross_entropy_with_logits(logits, targets, reduction='none')
        probs = torch.sigmoid(logits)
        pt = torch.where(targets == 1, probs, 1 - probs)
        focal_term = (1 - pt) ** self.gamma
        loss = focal_term * bce_loss

        if self.alpha is not None:
            alpha_factor = torch.where(targets == 1, self.alpha, torch.ones_like(targets))
            loss = alpha_factor * loss

        return loss.mean()
```

pos_weights 计算：

```
for i in range(1, NUM_ABNORMAL_CLASSES + 1):
    pos = label_counter.get(i, 0) # 正样本数
    neg = num_samples - pos # 负样本数
    pos_weights.append(min(max((neg / max(pos, 1)) ** 0.5, 1.0), 10.0))
```

使用类别频率的平方根加权，范围限制在 [1.0, 10.0]。

5.2 备选模型：MultiScaleFusionResNet

文件位置： models/fs_resnet.py

基于 ResNet50 的骨架注入模型，作为 ConvNeXt 的备选方案：

```

class MultiScaleFusionResNet(nn.Module):
    def __init__(self, num_classes=8, input_size=224):
        super().__init__()
        res50 = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)

        # 拆解 ResNet50
        self.conv1 = res50.conv1
        self.bn1 = res50.bn1
        self.relu = res50.relu
        self.maxpool = res50.maxpool
        self.layer1 = res50.layer1 # 56x56
        self.layer2 = res50.layer2 # 28x28
        self.layer3 = res50.layer3 # 14x14
        self.layer4 = res50.layer4 # 7x7

        # 骨架注入层
        self.attn1 = SkeletonAttention(256, sk_dim=2) # layer1 后
        self.attn2 = SkeletonAttention(512, sk_dim=2) # layer2 后
        self.attn3 = SkeletonAttention(1024, sk_dim=2) # layer3 后
        self.attn4 = SkeletonAttention(2048, sk_dim=2) # layer4 后

        self.fc = nn.Sequential(
            nn.Dropout(0.8),
            nn.Linear(2048, num_classes)
        )

    def forward(self, x_5ch):
        img = x_5ch[:, :3, :, :] # RGB 通道
        sk_combined = x_5ch[:, 3:, :, :] # 骨架通道

        x = self.relu(self.bn1(self.conv1(img)))
        x = self.maxpool(x)

        # 每个 stage 后注入骨架信息
        x = self.layer1(x); x = self.attn1(x, sk_combined)
        x = self.layer2(x); x = self.attn2(x, sk_combined)
        x = self.layer3(x); x = self.attn3(x, sk_combined)
        x = self.layer4(x); x = self.attn4(x, sk_combined)

        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        return self.fc(x)

```

SkeletonAttention 模块：

```

class SkeletonAttention(nn.Module):
    def __init__(self, feature_dim, sk_dim=2):
        super().__init__()
        self.query_conv = nn.Conv2d(feature_dim, feature_dim // 8, 1)
        self.key_conv = nn.Conv2d(sk_dim, feature_dim // 8, 1)
        self.value_conv = nn.Conv2d(sk_dim, feature_dim, 1)
        self.gamma = nn.Parameter(torch.zeros(1))

    def forward(self, x, sk_map):
        sk_resized = F.interpolate(sk_map, size=(h, w), mode='bilinear')
        # Q 来自特征图, K/V 来自骨架图
        # 计算注意力并注入
        return x + self.gamma * out

```

5.3 模型参数分类与学习率

参数组	包含模块	学习率	说明
backbone_params	ConvNeXt 主干网络（除 stem、attn、fusion）	1e-4	预训练权重，小学习率
head_params	classifier, head	1e-4	随机初始化的分类头，较大学习率
inject_params	attn, stem_conv	1e-4	新增的注入层
fusion_params	fusion_logits	5e-3	融合权重，最大学习率

优化器配置：

```
optimizer = optim.Adam([
    {"params": fusion_params, "lr": 5e-3},
    {"params": backbone_params, "lr": 10e-5},
    {"params": head_params,      "lr": 10e-5},
    {"params": inject_params,    "lr": 10e-5},
], weight_decay=3e-3)
```

6. 训练流程（sk_resnet.py）

6.1 训练配置

文件位置： sk_resnet.py

```
# 数据路径
train_dir = 'data/rearranged_train' # 50,459 张
val_dir   = 'data/rearranged_val'   # 14,417 张
test_dir  = 'data/rearranged_test'  # 7,209 张

# 训练参数
batch_size = 64
num_epochs = 40
input_size = 224
NUM_ABNORMAL_CLASSES = 8
target_T_number = 100 # 异常样本目标数量（用于平衡评估）
target_F_number = 100 # 正常样本目标数量
```

6.2 数据加载

```
train_transform = JointTransform(  
    input_size=224,  
    hflip_prob=0.5,  
    rotate_deg=5,  
    rotate_prob=0.5,  
    brightness=0.05,  
    contrast=0.05,  
    brightness_prob=0.5,  
    contrast_prob=0.5  
)  
  
val_transform = JointTransform(  
    input_size=224,  
    hflip_prob=0.0, # 验证集不做增强  
    rotate_deg=0,  
    rotate_prob=0.0,  
    brightness=0.0,  
    contrast=0.0,  
    brightness_prob=0.0,  
    contrast_prob=0.0  
)  
  
train_dataset = ImageWithSKMultiLabelDataset(  
    root_dir=train_dir,  
    transform=train_transform,  
    num_classes=NUM_ABNORMAL_CLASSES,  
    input_size=input_size  
)  
  
# Dataloader  
train_loader = DataLoader(  
    train_dataset,  
    batch_size=batch_size,  
    shuffle=True,  
    num_workers=8,  
    pin_memory=True  
)
```

6.3 训练流程

```
for epoch in range(num_epochs):
    model.train()

    # 前 free_epochs(8) 轮冻结 backbone
    if epoch == free_epochs or load_parameter:
        for p in backbone_params:
            p.requires_grad = True

    for images, labels_multi, labels_binary in tqdm(train_loader):
        images = images.to(device)
        labels_multi = labels_multi.to(device)
        labels_binary = labels_binary.to(device)

        optimizer.zero_grad()

        # 前向传播
        output = model(images)

        # 计算损失
        loss_multi, loss_binary, loss_patch, total_loss = calc_all_loss(
            output, labels_multi, labels_binary,
            criterion_multi, criterion_binary,
            loss_binary_para=5.0,    # 二分类损失权重
            loss_patch_para=0.0     # patch 损失未启用
        )

        # 反向传播
        total_loss.backward()
        optimizer.step()

        # 更新 EMA 模型
        ema_model.update_parameters(model)

    # 学习率调度
    if epoch < warmup_epochs(10):
        warmup_scheduler.step()
    else:
        cosine_scheduler.step()

    # 验证和保存
    val_loss = eval_and_save()
```

6.4 训练策略详解

策略	说明	参数
Backbone 冻结	前 8 个 epoch 冻结 backbone，仅训练分类头和注入层	free_epochs=8
Warmup	前 10 个 epoch 线性增加学习率	warmup_epochs=10
Cosine Annealing	10 个 epoch 后余弦退火	T_max=40, eta_min=3e-5
EMA	维护模型权重的指数移动平均	decay=0.999
多损失联合优化	Focal Loss 解决类别不平衡	gamma=2.5, loss_binary=5.0
梯度缩放	使用 AMP (autocast + GradScaler) 防止梯度溢出	混合精度训练

Warmup 函数：

```
def warmup_then_const(epoch):
    if epoch < warmup_epochs:
        return (epoch + 1) / warmup_epochs # 0.1 → 1.0
    return 1.0
```

6.5 DDP 多卡训练

```
if use_ddp:
    import torch.distributed as dist
    dist.init_process_group(backend="nccl")
    local_rank = int(os.environ["LOCAL_RANK"])
    torch.cuda.set_device(local_rank)
    device = torch.device("cuda", local_rank)

    model = torch.nn.parallel.DistributedDataParallel(
        model,
        device_ids=[local_rank],
        output_device=local_rank,
        find_unused_parameters=False
    )

    # DDP Sampler
    train_sampler = DistributedSampler(train_dataset)
    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        sampler=train_sampler,
        num_workers=8,
        pin_memory=True
    )
```

DDP 训练命令：

```
torchrun --nproc_per_node=4 sk_resnet.py # 4 卡训练
```

6.6 缓存预生成

训练开始前批量生成所有骨架缓存：

```
batch_pre_generate_all_caches(train_dataset, mode='both')
batch_pre_generate_all_caches(val_dataset, mode='both')
batch_pre_generate_all_caches(test_dataset, mode='both')
```

6.7 保存的文件

文件	说明
best_model_opt.pth	验证集损失最优的普通模型权重
best_model_ema_opt.pth	验证集损失最优的 EMA 模型权重（推荐使用）
last_model_opt.pth	最后一轮模型权重

模型保存路径： models_parameter/resnet/

6.8 训练输出

每个 epoch 输出：

Epoch [1/40] Train Loss: 0.5678 Val Loss: 0.4321 EMA Val Loss: 0.4200
Train Acc: 0.8500 Val Acc: 0.8200 EMA Val Acc: 0.8300
Train Recall: 0.8600 Val Recall: 0.8400 EMA Val Recall: 0.8500
Train Precision: 0.8400 Val Precision: 0.8300 EMA Val Precision: 0.8400
--- Feature Fusion Weights ---
Avg Pool Weight (Alpha): 0.5234
Max Pool Weight (Beta): 0.4766

LR: 0.000034

7. 域自适应微调（domain_fine_tuning.py）

7.1 微调目标

针对跨域场景（从 HumanRefiner 数据集迁移到华为域数据），提供域自适应微调功能，防止灾难性遗忘。

7.2 微调策略

策略	说明
低学习率	3e-5，避免破坏预训练权重
混合数据采样	30% 源域 + 70% 目标域
Backbone 冻结	前 5 个 epoch 冻结，防止灾难性遗忘
EMA 模型	持续维护权重的移动平均 (decay=0.999)
Focal Loss	gamma=2.5，关注难样本

7.3 DomainFineTuner 类

```
class DomainFineTuner:
    def __init__(self, model, device='cuda', lr=3e-5, weight_decay=3e-3, ema_decay=0.999):
        # 分层学习率
        self.optimizer = optim.Adam([
            {"params": fusion_params, "lr": lr * 10},    # 5e-4
            {"params": backbone_params, "lr": lr},      # 3e-5
            {"params": head_params, "lr": lr},          # 3e-5
            {"params": inject_params, "lr": lr},        # 3e-5
        ], weight_decay=weight_decay)

        self.criterion_multi = FocalLoss(gamma=2.5)
        self.criterion_binary = FocalLoss(gamma=2.5)

        self.ema_model = AveragedModel(model, avg_fn=lambda averaged, current, num:
                                         ema_decay * averaged + (1 - ema_decay) * current)
```


7.4 微调流程

```
# 1. 加载预训练模型
model = ConvNextTinyFusion(num_classes=8, input_size=224)
model.load_state_dict(torch.load('models_parameter/resnet/best_model_ema_opt.pth'))

# 2. 加载数据
# 源域: rearranged_train (HumanRefiner 数据, 50,459 张)
# 目标域: huawei_data/train (华为域数据, 248 张)

# 3. 创建混合数据加载器
# 采样比例: 源域 30%, 目标域 70%
source_samples = max(1, int(len(source_train_dataset) * 0.3)) # 15,138
target_samples = max(1, int(len(target_train_dataset) * 0.7)) # 174

source_loader = DataLoader(source_train_dataset, batch_size=32, sampler=source_sampler)
target_loader = DataLoader(target_train_dataset, batch_size=32, sampler=target_sampler)

# 4. 执行微调
fine_tuner = DomainFineTuner(model, device=device, lr=3e-5, weight_decay=3e-3, ema_decay=0.999)

fine_tuner.fine_tune(
    source_loader=source_loader,
    target_loader=target_loader,
    num_epochs=15,
    save_path='models_parameter/finetuned/cross_domain_fine_tuned.pth',
    freeze_backbone=True,
    free_epochs=5,
    source_weight=0.3,
    target_val_loader=target_val_loader,
    batch_size=64
)
```

7.5 混合数据采样逻辑

```
# 每个 epoch 采样固定次数
epoch_length = max(len(source_loader), len(target_loader))

for _ in range(epoch_length):
    use_source = random.random() < source_weight # 30% 概率采样源域

    if use_source:
        images, labels_multi, labels_binary = next(source_iter)
    else:
        images, labels_multi, labels_binary = next(target_iter)
```

7.6 微调参数

参数	值	说明
lr	3e-5	低学习率防止灾难性遗忘
weight_decay	3e-3	正则化系数
num_epochs	15	微调轮数
freeze_backbone	True	冻结 backbone
free_epochs	5	前 5 轮冻结 backbone
source_weight	0.3	源域数据在混合数据中的比例
ema_decay	0.999	EMA 衰减率

参数	值	说明
batch_size	64	批次大小

7.7 保存的文件

文件	说明
models_parameter/finetuned/cross_domain_fine_tuned.pth	域自适应微调后权重

7.8 华为数据集使用说明

数据集	路径	用途	当前状态
源域数据	data/rearranged_train	预训练，作为基础域	已就绪，含 8 类异常标签
目标域训练集	huawei_data/train	域自适应微调的训练数据	需要做好异常类型标注
目标域测试集	huawei_data/test	微调后的评估数据	需要补充对应的图像和标签

注意： 华为数据集用于 fine-tune 部分，当前需要做好数据的异常类型标注（8 类异常：手部畸形、肢体缺失/增加、肢体/脸部模糊、脸部畸形、身体畸形等），以提升跨域性能。标注完成后即可用于域自适应微调，进一步提升在目标域上的精度和召回率。

8. 测试评估（eval.py）

8.1 评估流程

```
# 1. 加载微调后的模型
model = ConvNeXtTinyFusion(num_classes=8, input_size=224)
model.load_state_dict(
    torch.load('models_parameter/finetuned/cross_domain_fine_tuned.pth'),
    strict=False
)

# 2. 加载验证集和测试集
val_dir = 'data/huawei_data/train'      # 验证集（微调域）
test_dir = 'data/huawei_data/test'     # 测试集

# 3. 收集验证集 logits，优化阈值
all_binary_logits = []
all_binary_labels = []
with torch.no_grad():
    for images, labels_multi, labels_binary in val_loader:
        output = model(images)
        all_binary_logits.append(output["binary_logit"].cpu())
        all_binary_labels.append(labels_binary.cpu())

optimal_thresh, _ = find_optimal_threshold(all_binary_logits, all_binary_labels)
print(f"Optimal binary classification threshold on val set: {optimal_thresh:.4f}")

# 4. 评估验证集
val_acc, val_recall, val_precision = calc_metrics_balance(val_I, val_F, target_I, target_F, val_TT, val_TF, val_FT, val_FF)

# 5. 评估测试集
test_acc, test_recall, test_precision = calc_metrics_balance(...)
```

8.2 阈值优化

```
def find_optimal_threshold(logits, targets, min_thresh=0.2, max_thresh=0.8, step=0.005):
    """
    寻找最优的分类阈值，最大化 Balanced Accuracy + Recall。

    Args:
        logits: 模型二分类 logits [N]
        targets: 二分类 ground truth [N]
        min_thresh: 最小阈值 (0.2)
        max_thresh: 最大阈值 (0.8)
        step: 步长 (0.005)

    Returns:
        best_threshold: 最优阈值
        results: 每个阈值下的详细指标
    """
    thresholds = np.arange(min_thresh, max_thresh, step)
    probs = torch.sigmoid(logits).cpu().numpy()
    targets = targets.cpu().numpy()

    best_thresh = 0.5
    opt_score = -1

    for t in thresholds:
        preds = (probs > t).astype(int)
        TT, TF, FT, FF = calc_tf(preds, targets)
        acc, recall, _ = calc_metrics_balance(total_T, total_F, 100, 100, TT, TF, FT, FF)

        current_score = acc + recall
        if current_score > opt_score:
            opt_score = current_score
            best_thresh = t

    return best_thresh, results
```

8.3 保存预测结果

```
output_dir = test_dir + "/output_label"
os.makedirs(output_dir, exist_ok=True)

with torch.no_grad():
    for i, (images, labels_multi, labels_binary) in enumerate(test_loader):
        output = model(images)
        probs = torch.sigmoid(output["binary_logit"]).cpu()
        pred_labels = (probs >= optimal_thresh).int()

        for j in range(images.size(0)):
            img_name = test_loader.dataset.image_files[global_idx]
            filename = os.path.splitext(img_name)[0] + ".txt"
            output_path = os.path.join(output_dir, filename)

            with open(output_path, 'w') as f:
                f.write(str(probs[j].item()))
            global_idx += 1
```

8.4 概率分布可视化

```
def plot_prob_dist(logits, targets, name):
    plt.figure()
    probs = torch.sigmoid(logits).cpu().numpy()
    pos_probs = probs[targets == 1]
    neg_probs = probs[targets == 0]

    plt.hist(pos_probs, bins=50, alpha=0.5, label='Abnormal', color='red')
    plt.hist(neg_probs, bins=50, alpha=0.5, label='Normal', color='blue')
    plt.legend()
    plt.title("Probability Distribution_" + name)
    plt.savefig("probability_distribution_" + name + ".png")
```

9. 推理调试 (debug.py)

9.1 推理调试功能

debug.py 可以可视化模型推理的中间过程，包括：

子图编号	内容	说明
1	RGB 输入	原始输入图像（224x224）
2	人体骨架热力图	输入的第 3 通道
3	手部骨架热力图	输入的第 4 通道
4	Stem 输出	最大激活值（96 通道，56x56 → 224x224）
5	Stage 1 输出	最大激活值（96 通道，56x56 → 224x224）
6	Stage 2 输出	最大激活值（192 通道，28x28 → 224x224）
7	Stage 3 输出	最大激活值（384 通道，14x14 → 224x224）
8	Stage 4 输出	最大激活值（768 通道，7x7 → 224x224）
9	空间注意力图	CBAM 空间注意力（7x7 → 224x224）
10	Patch Score S4	补丁级异常分数（S4 层）
11	Patch Score S3	补丁级异常分数（S3 层）

9.2 推理流程

```
# 1. 加载模型
model = ConvNxtTinyFusion(num_classes=8, input_size=224)
model.load_state_dict(torch.load('models_parameter/resnet/best_model_ema_opt.pth'))
model.eval()

# 2. 加载图片并生成 5 通道输入
image_path = './debug/a_223.png'
x_5ch = load_image_and_create_5ch(image_path, sk_generator=use_mmpose_sk, input_size=224, device=device)

# 3. 执行推理调试
debug_inference(model, x_5ch, device=device)
```

9.3 输出示例

```
Binary Logit: 3.0070
Binary Probability: tensor([[0.9529]])
Multi Logit Mean: -1.6978
```

结果解读：

- 输入图片被判断为异常，异常概率 95.29%
- 推理调试图保存为 `./debug/detailed_debug_result.png`

10. 实际测试结果

10.1 测试环境

- **模型权重：** `models_parameter/finetuned/cross_domain_fine_tuned.pth` （域自适应微调后）
- **验证集：** `data/huawei_data/train` （248 张）
- **测试集：** `data/huawei_data/test` （248 张）

10.2 验证集结果

```
Optimal binary classification threshold on val set: 0.5000

Final Evaluation with Optimal Threshold 0.5000:
Val Loss: 1.1389
Val Acc: 1.0000
Val Recall: 1.0000
Val Precision: 1.0000
```

10.3 测试集结果

```
Test Evaluation with Optimal Threshold 0.5000:
Test Loss: 2.2994
Test Acc: 0.9156
Test Recall: 0.9512
Test Precision: 0.8880
```

10.4 结果分析

指标	验证集	测试集	说明
准确率 (Accuracy)	100.00%	91.56%	测试集准确率略低，符合预期
召回率 (Recall)	100.00%	95.12%	能找出 95% 以上的异常样本
精确率 (Precision)	100.00%	88.80%	误检率约 11%
损失 (Loss)	1.1389	2.2994	测试集损失较高，存在域差异

11. 完整复现步骤

11.1 环境准备

```
# 1. 创建 conda 环境
conda create -n aigc python=3.10 -y
conda activate aigc

# 2. 安装 PyTorch (根据 CUDA 版本调整)
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121

# 3. 安装 OpenMMLab 系列
pip install mmpose==0.10.7
pip install mmdet==3.3.0
pip install mmcv==2.1.0

# 4. 安装 Segment Anything
pip install git+https://github.com/facebookresearch/segment-anything.git

# 5. 其他依赖
pip install numpy Pillow matplotlib tqdm pyyaml opencv-python

# 6. 克隆/下载项目
git clone <project_url>
cd aigcAnomalyDetect

# 7. 下载 SAM 权重 (手动)
wget <sam_weight_url> -O sam_vit_h_4b8939.pth

# 8. 准备数据
# 将原始 HumanRefiner 数据集放置在 data/ 目录下
```

11.2 数据准备

```
# 1. 数据清洗
python dataWash.py # 标签合并, 生成 washed_labels/ 和 washed_multiple_labels/

# 2. 数据集重组
python rearrange_data.py # 合并所有数据, 7:2:1 划分, 软链接重组

# 3. 验证数据集
python -c "
from sk_dataset import ImageWithSKMultiLabelDataset, JointTransform
ds = ImageWithSKMultiLabelDataset('data/rearranged_train',
    JointTransform(input_size=224, hflip_prob=0.0), num_classes=8, input_size=224)
print(f'Train: {len(ds)} images')
"
```

11.3 主模型训练

```
# 单卡训练
python sk_resnet.py

# 多卡训练 (DDP)
torchrun --nproc_per_node=4 sk_resnet.py

# 训练完成后, 模型权重保存在:
# - models_parameter/resnet/best_model_ema_opt.pth (推荐使用)
# - models_parameter/resnet/best_model_opt.pth
# - models_parameter/resnet/last_model_opt.pth
```

训练时间估算：

- 单卡 (RTX 3090): 约 2-4 小时
- 4 卡 (RTX 3090): 约 1 小时

11.4 域自适应微调

```
# 执行域自适应微调
python domain_fine_tuning.py

# 微调完成后，模型权重保存在：
# - models_parameter/finetuned/cross_domain_fine_tuned.pth
```

11.5 测试评估

```
# 运行评估脚本
python eval.py

# 输出：
# - 验证集和测试集的准确率、召回率、精确率
# - 最优分类阈值
# - 测试集预测结果（每个样本的异常概率）
# - 概率分布直方图
```

11.6 推理调试

```
# 运行推理调试
python debug.py

# 修改 debug.py 中的图片路径以测试不同图片：
# image_path = './debug/a_223.png' # 替换为你的图片路径

# 输出：
# - 推理调试图：./debug/detailed_debug_result.png（11 个子图）
# - 推理结果：Binary Probability、Multi Logit Mean
```

11.7 合成数据（可选）

```
# 运行合成数据生成
python sam_fusion.py

# 或使用参考实现
python fusion_test/fusion.py

# 合成数据输出到：
# - data/fusion_result_1/washed_images/
# - data/fusion_result_2/washed_images/
# - data/fusion_result_3/washed_images/
```

11.8 使用训练好的模型进行推理

```
from models.ConvNeXt import ConvNeXtTinyFusion
from mmpose_sk import build_skeleton_mask, build_hand_mask
import torch
from PIL import Image
import torchvision.transforms.functional as TF

class AIGCAnomalyDetector:
    def __init__(self, model_path, device="cuda"):
        """加载模型"""
        self.model = ConvNeXtTinyFusion(num_classes=8, input_size=224)
        self.model.load_state_dict(torch.load(model_path, map_location=device), strict=False)
        self.model.to(device)
        self.model.eval()
        self.device = device

    def preprocess(self, image):
        """预处理: 生成 5 通道输入"""
        # RGB 归一化
        rgb = TF.to_tensor(image)
        rgb = TF.normalize(rgb, mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])

        # 骨架热力图
        skeleton = build_skeleton_mask(str(image), out_size=(224, 224)).to(self.device)
        hand_skeleton = build_hand_mask(str(image), out_size=(224, 224)).to(self.device)

        # 拼接 5 通道
        x_5ch = torch.cat([rgb, skeleton, hand_skeleton], dim=0).unsqueeze(0)
        return x_5ch

    def predict(self, image):
        """执行推理"""
        x_5ch = self.preprocess(image)
        with torch.no_grad():
            output = self.model(x_5ch)

        return output

# 使用示例
detector = AIGCAnomalyDetector('models_parameter/resnet/best_model_ema_opt.pth')
image = Image.open('test_image.jpg').convert('RGB')
result = detector.predict(image)

print(f"异常概率: {torch.sigmoid(result['binary_logit']).item():.4f}")
print(f"8 类异常分数: {result['multi_logits'].sigmoid().cpu().numpy()}")
```

附录 A：配置文件（config.yaml）

```
data_dir: /home/lwx1502137/aigcdetect/aigcAnomalyDetect/data
model_dir: /home/lwx1502137/aigcdetect/aigcAnomalyDetect/models_parameter/resnet
finetuned_dir: /home/lwx1502137/aigcdetect/aigcAnomalyDetect/models_parameter/finetuned
root_dir: /home/lwx1502137/aigcdetect/aigcAnomalyDetect
my_test_dir: /home/lwx1502137/aigcdetect/aigcAnomalyDetect/labeled_data
target_T_number: 100      # 异常样本目标数量（用于平衡评估）
target_F_number: 100      # 正常样本目标数量
input_size: 224           # 输入图像尺寸
use_ddp: False           # 是否使用 DDP
```


附录 B：关键文件索引

文件	功能
sk_resnet.py	主训练脚本（训练 + EMA + 评估 + 保存）
domain_fine_tuning.py	域自适应微调脚本（DomainFineTuner 类）
eval.py	独立评估脚本（阈值优化 + 测试集评估 + 概率分布可视化）
debug.py	推理调试脚本（可视化中间过程）
sk_dataset.py	数据集类 + JointTransform + FocalLoss + LabelSmoothingBCEWithLogitsLoss
loss_calculator.py	损失函数计算器（多任务损失 + 阈值优化 find_optimal_threshold）
evaluate/evaluate.py	评估指标计算（calc_metrics_balance, calc_tf）
evaluate/ml_evaluate.py	多标签评估（evalModel_multiple）
models/ConvNeXt.py	主模型 ConvNeXtTinyFusion（含 SpatialAttention, StandardCrossAttention, PatchAnomalyHead）
models/fs_resnet.py	备选模型 MultiScaleFusionResNet（基于 ResNet50）
mmpose_sk.py	人体/手部关键点检测 + 骨架热力图生成（RTMPose-L Body, RTMPose-M Hand）
sam_fusion.py	SAM 手部 patch 融合到目标图（完整 pipeline）
dataWash.py	数据集清洗脚本（find_label_files, find_multiple_label_files, label_count）
rearrange_data.py	数据集合并重组（7:2:1 划分，软链接）
rearrange_finetune_data.py	微调数据集重组
sam.py	SAM 资产库构建脚本
fusion_test/fusion.py	合成数据参考实现（含仿射变换和对齐逻辑）
config.yaml	全局配置文件
sam_vit_h_4b8939.pth	SAM ViT-H 模型权重（2.5 GB）

附录 C：依赖版本详情

```
PyTorch: 2.4.0+cu121
torchvision: >= 0.19
MMPose: 0.10.7
MMDetection: 3.3.0
MMCV: 2.1.0（mmpose_sk.py 中伪装为 2.1.0）
Python: 3.10
CUDA: 可用
NumPy: -
Pillow: -
Matplotlib: -
OpenCV: -
tqdm: -
PyYAML: -
```

附录 D：模型参数量与训练资源

指标	值	说明
模型参数量	~28M	ConvNeXt-Tiny 级别，轻量级模型
输入尺寸	224×224	固定输入尺寸
输入通道	5	RGB(3) + 人体骨架(1) + 手部骨架(1)
特征维度	4520	9 路特征融合后的总维度
分类头输出	9	8(多标签) + 1(二分类)
显存占用 (batch=1)	~4GB	GPU 显存
显存占用 (batch=32)	~6GB	GPU 显存
训练速度 (单卡)	约 50 秒/epoch	RTX 3090, batch=64
推理速度 (单张)	200-500 张/秒	RTX 3090/A100
推理速度 (batch=32)	800-1500 张/秒	RTX 3090/A100

报告结束